Source-based confirmation does not establish runtime correctness, design quality or completed human verification. Preserve the controller-heavy and transaction-consistency concerns separately from the pattern classification.

Final conclusion: Both candidates meet the GRASP Controller responsibility rule at the source level, with different strengths and design trade-offs. They can be proposed as two distinct baseline instances, subject to human verification and consistent counting across the rest of the repository.

Now move to the next source area: itinerary-item creation, imported record linking and expense handling.
As before, do not begin by assigning pattern names. Trace the actual collaborations first.
Inspect these files and any directly related callers needed to complete the flows:

1. `app/controllers/trip_items_controller.rb`
2. `app/models/trip_item.rb`
3. `app/models/inbox_item.rb`
4. `app/models/expense.rb`
5. `app/controllers/expenses_controller.rb`
6. `app/jobs/parse_receipt_job.rb`
7. Relevant views, routes and model associations where needed.

Some `InboxItem` behavior was already examined during the inbound-email investigation. Reuse established evidence where valid, but do not count the same collaboration again under a new name.
I want to understand these areas separately.

1. Manual itinerary-item creation and editing.

Trace how a user creates, updates or removes a `TripItem`, including any model behavior or related expense handling.

2. Bulk itinerary import.

Trace the import path and `bulk_create_items` behavior. Identify how records are created, how geocoding is handled and what logic is application-written versus Rails behavior.

3. Inbox-derived record linking.

Use the already traced `InboxItem#fill_missing_links!` and related methods only where they interact with this source area. Determine whether this reveals a genuinely new collaboration or is simply the previously confirmed Parsed Email to Trip Record Linking instance.

4. Expense creation and management.

Trace normal expense creation/editing and any automatic expense creation connected to itinerary items or inbox data.

5. Receipt parsing.

Trace exactly what `ParseReceiptJob` receives, what external service or library it uses, what information it extracts and how that result changes an `Expense`.
For each flow:

1. Identify the actual participating classes and methods.
2. Give reliable source paths and line ranges.
3. Trace the control flow and important decisions.
4. Separate application-owned behavior from Rails and external-provider behavior.
5. Identify where responsibilities are assigned and delegated.
6. Check whether apparently similar creation paths actually share one collaboration or represent independent responsibilities.
7. Do not count the already confirmed `Parsed Email to Trip Record Linking` instance again merely because `TripItem` and `Expense` appear here.

Only after tracing the flows, identify any genuinely new design-pattern candidates.
For each candidate:

1. State the possible pattern and category.
2. Give a meaningful collaboration name.
3. Identify the actual participants.
4. Explain what behavior supports the classification.
5. Give the strongest false-positive explanation.
6. State whether the behavior is application-owned, framework-provided or mixed.
7. Leave the candidate UNCONFIRMED.

Do not force Factory, Builder, Strategy, Command, Template Method, Adapter, Creator or Information Expert unless the defining mechanism or responsibility is actually supported.
At the end give me:

1. Manual TripItem flow.
2. Bulk-import flow.
3. Inbox-linking overlap analysis.
4. Expense-management flow.
5. Receipt-parsing flow.
6. Participant/source-evidence table.
7. New candidate patterns only.
8. Previously confirmed patterns encountered again but not counted again.
9. Strongest non-pattern explanation.
10. Anything that still requires verification.

Worked for 1m 57s

DaPlanStan — Itinerary Items, Imported Linking and Expenses

Candidates unconfirmed Source Investigation 04 · Original uploaded ZIP

1. Initial findings

This source area contains several paths for creating and updating travel data, but they do **not** all belong to one collaboration.

- Manual itinerary management uses `TripItemsController` and `TripItem`.
- Bulk JSON import uses a separate controller method with duplicate-name checking and queued geocoding.
- Email-derived linking uses the previously examined `InboxItem` methods.
- Expenses can be entered manually, prefilled from itinerary items or created from confirmed email data.
- Receipt parsing uses a separate background job that invokes Anthropic and updates an existing `Expense`.

A particularly important distinction is that `TripItemsController#to_expense` **does not create an expense**. It prepares values and redirects to the expense form.

Similarly, `ParseReceiptJob` does not create an expense. It operates on an expense that already exists.

2. Manual itinerary-item creation and editing